

تمرین کامپیوتری شماره 3

سید امین اسماعیلی زکی 810103356

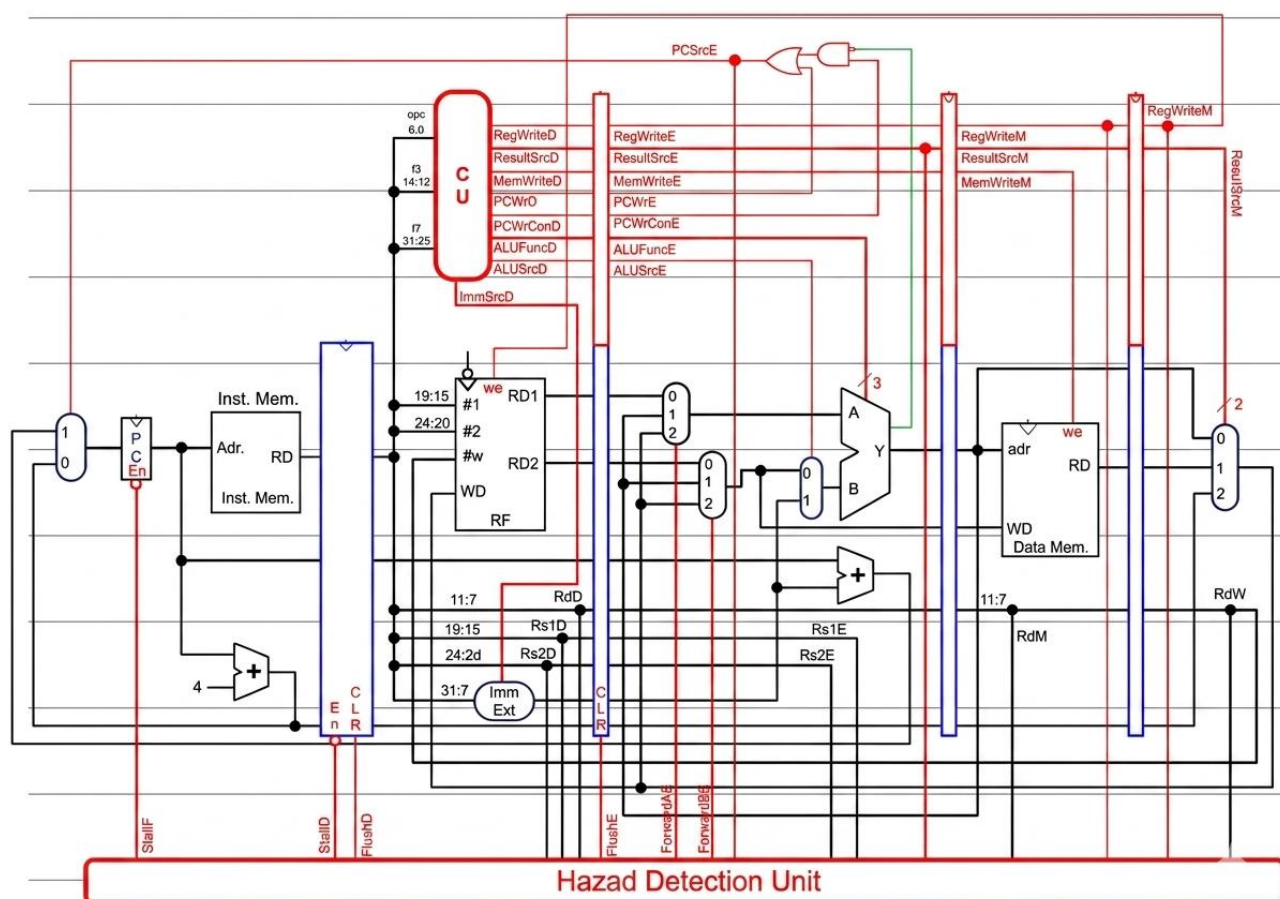
810103414 علی دهقانی دهقی

طراحی مسیر داده:

1. مسیر داده پایه:

- برای دستورات زیر از آخرین مثال موجود در اسلاید های تدریس استاد استفاده می کنیم (که مسیر داده آن به تمام آنها را پشتیبانی می کند):

- R-Type: add, sub, and, or, slt
- I-Type: lw, addi, ori, slti
- S-Type: sw
- J-Type: jal
- B-Type: beq



- دستورات R-Type مستقیماً از طریق ALU قابل پیاده سازی هستند.
- همچنین برای دستورات I-Type به جز lw سه مورد دیگر از طریق Imm Ext و توابع ALU قابل اجرا هستند.
- دستور jal هم از طریق مسیر آبی رنگ که در همین مثال پیاده سازی شد قابل اجرا است.
- دستورات beq, sw, lw هم از دستورات پایه هستند که از قبل هم در مسیرهاده وجود داشتند.

2. اضافه کردن دستورات جدید:

- I-Type: xori

- برای دستور بالا مجبور هستیم که یک تابع جدید به ALU اضافه کنیم چون با توابع کنونی آن برای پیاده سازی تابع xor نیازمند بیش از یک کلاک هستیم و این با نحوه پیاده سازی ما (single-cycle) در تناقض است. پس تابع xor را به ALU اضافه می کنیم.

- I-Type: jalr

- تفاوت این دستور با دستور jal به تنها در نحوه مقداردهی pc است:

$$\begin{aligned} \text{jal rd, imm_20bit} &\longrightarrow \begin{cases} \text{rd} \leq \text{pc} + 4 \\ \text{pc} \leq \text{pc} + \text{imm_20bit} \end{cases} \\ \text{jal rd, rs, imm_12bit} &\longrightarrow \begin{cases} \text{rd} \leq \text{pc} + 4 \\ \text{pc} \leq \text{rs} + \text{imm_12bit} \end{cases} \end{aligned}$$

- در نتیجه برای پیاده سازی آن کافی است در مسیر محاسبه مقدار جدید برای pc که برای jal فراهم شده است یک مولتی پلکسر قرار بدهیم تا از بین خروجی رجیسترفایل (rs) و مقدار فعلی pc یکی را انتخاب و با خروجی Imm Ext جمع کند.

- B-Type: bne, blt, bge

- با توجه به اینکه از قبل دستور beq را داریم پیاده سازی آنها خیلی شبیه به آن و ساده خواهد بود:
- bne: کافی است برعکس شرط beq را بررسی کنیم یعنی در صورتی پرش را انجام دهیم که اختلاف دو عدد صفر نباشد و در نتیجه سیگنال zero برابر صفر باشد.
- blt: برای آن از تابع ALU slt استفاده میکنیم به این صورت اگر ورودی A کوچکتر از B باشد خروجی ALU یعنی Y برابر 1 می شود و در نتیجه سیگنال zero صفر می شود. پس میتوانیم براساس zero تصمیم گیری کنیم که اگر صفر بود پرش انجام شود.
- Bge: برعکس دستور قبل عمل میکنیم چون در این دستور اگر ورودی A بزرگتر یا مساوی B باشد باید پرش را انجام دهیم و اگر این اتفاق بیفتد خروجی Y در حالت slt برابر 0 می شود که باز هم میتوانیم از سیگنال zero استفاده کنیم که اگر یک باشد پرش را انجام دهیم.

3. کشیدن مسیر داده:

- طبق توضیحات بخش قبل تنها باید یک مولتی پلکسر به مسیر داده اضافه کنیم. بر روی ورودی دوم adder بالاتر که ورودی های آن pc و RD1 است، همچنین چند خطا در نامگذاری سیم ها و ورودی بخش تشخیص هازاد تصحیح می کنیم.

طراحی کنترلر:

1. ALUFunc:

ALU Func	Y
000	$A+B$ (add)
001	$A-B$ (sub)
010	$A \& B$ (and)
011	$A B$ (or)
100	$A \leq B$ (set less than)
101	$A \oplus B$ (xor)

- طبق توضیحاتی که قبل تر داده شد یک تابع جدید XOR به حالت های قبلی اضافه می کنیم:

2. ImmSrc:

- با توجه اینکه تایپ جدیدی از دستورات را اضافه نکرده ایم میتوانیم از همان حالت مثال جزوه برای استخراج مقدار فوری استفاده کنیم:

Imm Src	Imm Ext	Inst Type
00	$\{ \{ 20 \{ Inst[31] \} \}, Inst[31:20] \}$	I-Type (lw)
01	$\{ \{ 20 \{ Inst[31] \} \}, Inst[31:25], Inst[11:7] \}$	S-Type (sw)
10	$\{ \{ 19 \{ Inst[31] \} \}, Inst[31], Inst[7], Inst[30:25], Inst[11:8], 1'60 \}$	B-Type (bge)
11	$\{ \{ 12 \{ Inst[31] \} \}, Inst[19:12], Inst[20], Inst[30:21], 1'60 \}$	J-Type (jal)

3. شکل کلی کنترلر:

- با توجه به اضافه کردن تعداد زیاد دستور که باعث تغییر مقدار pc و عمل پرش (bne, blt, bge, jalr) می شوند در نتیجه باید برای تشخیص سیگنال مولتی پلکسر ورودی آن یک واحد مجزا در نظر بگیریم و سیگنال PCWrite را برای دو دستور jal و jalr اضافه کنیم و سیگنال PCWriteCond برای دستورات پرش شرطی.
- همچنین سیگنال دیگری برای برای مولتی پلکسری که به مسیر داده اضافه شد باید در نظر بگیریم.
- کنترلر از سه بخش تشکیل شده است که شامل:

- Main controller: بخش اصلی کنترلر است که تمام سیگنال های لازم را به جز ALUFunc ایجاد میکند.
- PCController: قسمت مجزا که تنها برای کنترل سیگنال لود رجیستر pc استفاده می شود و از دو سیگنال خروجی کنترلر اصلی استفاده میکند: PCWrite, PCWriteCond و همچنین خروجی ALU, zero. همچنین func3 را هم به عنوان ورودی دریافت می کند زیرا برای تصمیم گیری برای پرش های شرطی براساس نوع آن (که از روی func3 مشخص میشود) باید تصمیم بگیرد که از سیگنال zero به چه شکل استفاده کند (برای مثال برای beq اگر zero صفر بود پرش را انجام دهد ولی برای bne اگر صفر نبود).
- ALUController: با دریافت سیگنال ALUOP و func3 و func7 سیگنال ALUFunc را ایجاد میکند.
- لازم به ذکر است که بخش PC Controller را از دو بخش دیگر که در CU قرار دارند جدا میکنند زیرا تصمیم گیری آن باید در سیکل سوم (یا همان بخش ALU) باشد و دنتیجه سیگنال های مورد نیاز آن باید در یک بخش رجیستر شوند تا از سیگنال های مربوط به دستور صحیح بتواند استفاده کند.

4. جزئیات سیگنال های کنترل:

	op	op code	func3	func7	Result Src	Mem Write	ALU Src	Adder Src	Imm Src	Reg Write	PCWrite Cond	PCWrite	ALUop	PCSrc	ALUFunc
R_Type	add	0110011	000	00000000	00	0	0	X	XX	1	0	0	10	0	000
	sub		000	01000000	00	0	0	X	XX	1	0	0	10	0	001
	and		111	00000000	00	0	0	X	XX	1	0	0	10	0	010
	or		110	00000000	00	0	0	X	XX	1	0	0	10	0	011
I_Type	slt		010	00000000	00	0	0	X	XX	1	0	0	10	0	100
	lw	00000011	010	-	01	0	1	X	00	1	0	0	00	0	000
	addi	00100011	000		00	0	1	X	00	1	0	0	11	0	000
	xori		100		00	0	1	X	00	1	0	0	11	0	101
	ori		110		00	0	1	X	00	1	0	0	11	0	011
	slli		010		00	0	1	X	00	1	0	0	11	0	100
	jalr	11001111	000		10	0	X	1	00	1	0	1	XX	0	XXX
S_Type	sw	01000011	010	-	XX	1	1	X	01	0	0	0	00	0	000
J_Type	jal	11011111	-	-	10	0	X	0	11	1	0	1	XX	0	XXX
B_Type	beq	11000011	000	-	XX	0	0	0	10	0	1	0	01	zero	001
	bne		001		XX	0	0	0	10	0	1	0	01	~zero	001
	blt		100		XX	0	0	0	10	0	1	0	01	~zero	100
	bge		101		XX	0	0	0	10	0	1	0	01	zero	100

طراحی بخش تشخیص مخاطره:

1. مخاطره داده ای:

- طبق توضیحات استاد برای استفاده از بخش Data Forwarding باید از کدهای زیر استفاده کنیم:

```
if((RegWriteM == 1) and
    (Rs1E == RdM) and
    (Rs1E != 0))
    ForwardAE = 01;
else if((RegWriteW == 1) and
    (Rs1E == RdW) and
    (Rs1E != 0))
    ForwardAE = 10;
else
    ForwardAE = 00;
if((RegWriteM == 1) and
    (Rs2E == RdM) and
    (Rs2E != 0))
    ForwardBE = 01;
else if((RegWriteW == 1) and
    (Rs2E == RdW) and
    (Rs2E != 0))
    ForwardBE = 10;
else
    ForwardBE = 00;
```

- کد بالا برای همه ی دستوراتی که وابستگی ایجاد میکنند (مانند R-T و I-T کاربرد دارد و همه را به درستی اولویت بندی می کند ولی نکته ای که وجود دارد این است که دستور sw چون دیتای درست را در پایان سیکل M به دست می آورد به هر حال مجبور است پایپ لاین را یک سیکل stall کند و به همین منظور از سیگنال های زیر و به این شکل استفاده میکنیم:

```
lwStall = ((Rs1D == RdE) or (Rs2D == RdE)) and (ResultSrcE = 01);
stallF = lwStall
stallD = lwStall
flashE = lwStall
```

2. مخاطره کنترلی:

- تنها برای دستورات پرش چنین مخاطره ای به وجود می آید که دو دسته هستند:

- پرش شرطی: در این دستورات ما آدرس پرش را در همان سیکل اول می دانیم ولی باید شرط پرش بررسی شود و اگر برقرار بود پرش را انجام دهیم و این موضوع در پایان سیکل ALU دستور مشخص می شود و تا آن زمان باید صبر کنیم و با فرض برقرار نبودن شرط دستورات بعدی را وارد پایپ لاین می کنیم و در صورتی که شرط برقرار نباشد پیش بینی ما درست بوده است و اگر شرط غلط باشد پیش بینی ما غلط بوده و در واقع پایپ لاین دو سیکل stall شده و باید آن را فلش کنیم.

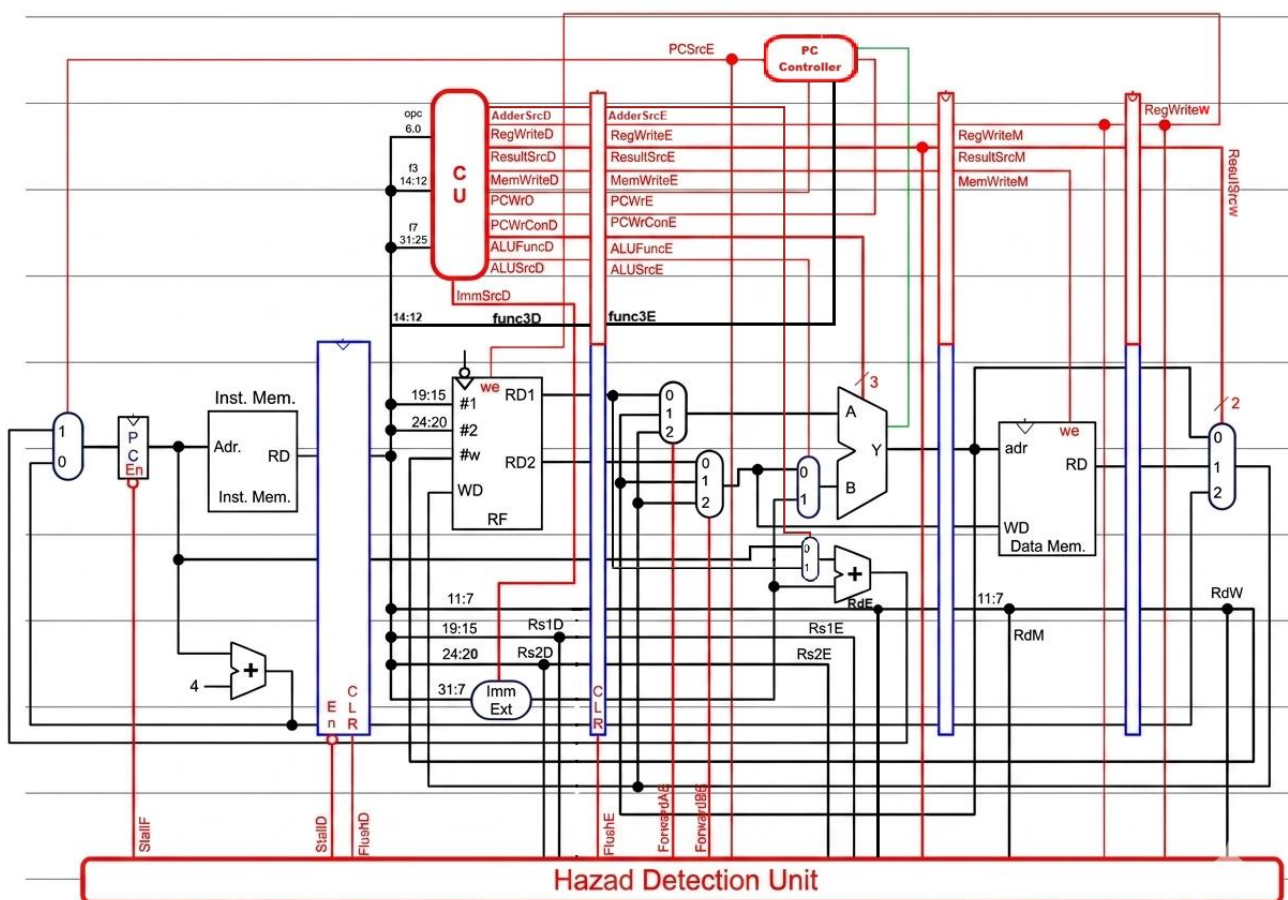
2) پرش قطعی: این دستورات در واقع همان دستورات JAL و JALR هستند که در سیکل اول دستور می دانیم باید پرش را انجام دهیم ولی نمی دانیم کجا باید پرش کنیم و آدرس پرش در پایان سیکل ALU دستور مشخص می شود پس مجبوریم دو سیکل پایپ لاین را stall کنیم و اصلاً نباید دستوری را وارد کنیم و یا اگر وارد کردیم حتماً باید فلیش شود چون برخلاف حالت قبل در اینجا پرش قطعی است.

○ حال برای فهمیدن اینکه پرش انجام شده یا نه برای هر دو حالت کافی است که سیگنال PCSrcE را بررسی کنیم چون در صورتی که آن مقدار 1 داشته باشد یعنی پرش انجام شده است و پایپ لاین باید برای دو بخش رجیستر اول فلیش شود چون دستورات ورودی معتبر نیست و از طریق کد زیر این کار قابل انجام است:

```
flashD = PCSrcE
flashE = lwStall or PCSrcE
```

شماتیک پردازنده:

○ با اعمال تمام تغییرات لازم طبق بخش های قبل به چنین نتیجه ای می رسیدیم:



پیاده سازی مسیر داده:

1. ماژول های تکراری:

برای پیاده سازی مسیر داده ابتدا باید ماژول ها موجود در آن را پیاده سازی کنیم. بسیاری از این ماژول ها در پروژه اول و دوم طراحی شده اند و نیازی به طراحی مجدد ندارند:

- ALU
- Data_Mem
- Inst_Mem
- Imm_Ext
- Multiplexer2to1
- Multiplexer3to1
- Sum

مانند پروژه اول و دوم، در اینجا صرفاً آن ها را بیان می کنیم و در تاپ ماژول آن ها را به پردازنده RISCv متصل می کنیم.

2. ماژول با تغییرات کم:

ماژول RF نیاز است تا مقداری در آن تغییر ایجاد شود تا بتوان از آن استفاده کرد. در این پروژه برخلاف قبلی و طبق طراحی این ماژول با لبه پایین رونده کلاک تغییر می کند. پس برای این کار این ماژول را به صورت زیر طراحی می کنیم:

```
module RF
(
    ...
);
always @(negedge clk)
```

در این جا به خاطر negedge بودن ماژول دیگر به لبه پایین رونده کلاک کار می کند.

3. ماژول های جدید:

در این پروژه ما نیاز به منابع رجیستری داریم که خود حاوی رجیستر های دیگر باشند و بتوان به کمک آن ها مقادیر را برای مرحله بعدی ذخیره کرد. این کار هم برای مسیر داده و هم برای کنترلر است. نیاز ما از رجیستر های این بخش متفاوت است. ما برای IF_ID نیاز به رجیستر هایی داریم که هم ریست و هم فعال کننده دارند، برای IF_EX نیاز به فقط ریست داریم و برای آن دو آخری نیاز به هیچ کدام نداریم. برای پیاده سازی هم به حالت کامل، درون رجیستر هایمان هم ریست و هم اینیبل را قرار می دهیم و در صورت استفاده نداشتن مقدار آن ها را صفر و یک قرار می دهیم:

```
module Register
(
    input clk, reset, enable,
    input [31:0] reg_in,
    output reg [31:0] reg_out
);
```

```

always @(posedge clk)
begin
    if (reset)
        reg_out <= 32'd0;

    else if (enable)
        reg_out <= reg_in;
end
endmodule

```

بدین ترتیب ریز ماژول های ما برای این بخش می شود:

- Register
- Register5bit

به کمک این دو ما حافظه های میان مراحل مختلف را طراحی می کنیم:

- IF_ID
- ID_EX
- EX_MEM
- MEM_WB

که مثلاً درون IF_ID از تمام ورودی های رجیستر ها به صورت کامل استفاده می کنیم:

```
Register stage0 (clk, reset, enable, in0, out0);
```

درون ID_EX فقط از ریست استفاده می کنیم و enable را همیشه برابر 1 قرار می دهیم:

```
Register5bit stage0 (clk, reset, 1'b1, in0, out0);
```

و درون EX_MEM و MEM_WB ریست را برابر 0 و enable را برابر 1 قرار می دهیم:

```
Register5bit stage0 (clk, 1'b0, 1'b1, in0, out0);
```

4. اتصال ماژول ها:

تمام ماژول ها مورد نیاز برای مسیر داده دیگر ساخته شدند. برای ماژول PC هم از همان Register استفاده می کنیم و دیگر نیازی به ماژول جدید دیگری نیست. حال نیاز است که این ها را به هم متصل کنیم. دقت می کنیم که ماژول های حافظه را در این قسمت قرار نمی دهیم و موارد مورد نیاز از آن ها را ورودی و خروجی می دهیم. همچنین به قرینه های موجود برای ورودی های stall نیز توجه می کنیم که مشکلی به وجود نیاید. کد مسیر داده به صورت زیر در می آید:

```

module DataPath
(
    input clk, reset, PCSrc, AdderSrc, RegWrite, ALUSrc, StallF, StallD, FlushD,
    FlushE,
    input [1:0] ResultSrc, ImmSrc, ForwardAE, ForwardBE,
    input [2:0] ALUFunc,
    input [31:0] inst_RD, data_RD,
    output zero,

```

```

    output [4:0] Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW,
    output [31:0] inst_adr, data_adr, data_WD, inst
);

    wire [31:0] PC_in, PC_out_IF, PC_out_ID, PC_out_EX, sum0_out_IF, sum0_out_ID,
sum0_out_EX, sum0_out_MEM, sum0_out_WB, A, B, Y,
                sum1_out, RegWriteData, RegData1_ID, RegData2_ID, Imm_out_ID,
Imm_out_EX, RegData1_EX, RegData2_EX,
                mult5_out, mult3_out, data_adr_WB, data_RD_WB;

    wire [4:0] RdD;

    // IF
    Register PC (clk, reset, ~StallF, PC_in, PC_out_IF);
    assign inst_adr = PC_out_IF;
    Sum sum0 (PC_out_IF, 32'd4, sum0_out_IF);
    Multiplexer2to1 mult0 (sum0_out_IF, sum1_out, PCSrc, PC_in);

    // IF_ID
    IF_ID stage_IF_ID (clk, FlushD, ~StallD, inst_RD, PC_out_IF, sum0_out_IF,
                        inst, PC_out_ID, sum0_out_ID);

    // ID
    RF stage_RF (clk, inst[19:15], inst[24:20], RdW, RegWriteData, RegWrite,
RegData1_ID, RegData2_ID);
    Imm_Ext stage_Imm (inst[31:7], ImmSrc, Imm_out_ID);

    assign RdD = inst[11:7];
    assign Rs1D = inst[19:15];
    assign Rs2D = inst[24:20];

    // ID_EX
    ID_EX stage_ID_EX (clk, FlushE, RdD, Rs1D, Rs2D, RegData1_ID, RegData2_ID,
PC_out_ID, Imm_out_ID, sum0_out_ID,
                        RdE, Rs1E, Rs2E, RegData1_EX, RegData2_EX,
PC_out_EX, Imm_out_EX, sum0_out_EX);

    // EX
    ALU stage_ALU (A, B, ALUFunc, Y, zero);
    Sum sum1 (mult5_out, Imm_out_EX, sum1_out);

    Multiplexer3to1 mult2 (RegData1_EX, data_adr, RegWriteData, ForwardAE, A);
    Multiplexer3to1 mult3 (RegData2_EX, data_adr, RegWriteData, ForwardBE,
mult3_out);
    Multiplexer2to1 mult4 (mult3_out, Imm_out_EX, ALUSrc, B);
    Multiplexer2to1 mult5 (PC_out_EX, RegData1_EX, AdderSrc, mult5_out);

    // EX_MEM

```

```

EX_MEM stage_EX_MEM (clk, RdE, Y, mult3_out, sum0_out_EX,
                    RdM, data_adr, data_WD, sum0_out_MEM);

// MEM_WB
MEM_WB stage_MEM_WB (clk, RdM, data_adr, data_RD, sum0_out_MEM,
                    RdW, data_adr_WB, data_RD_WB, sum0_out_WB);

// WB
Multiplexer3to1 mult6 (data_adr_WB, data_RD_WB, sum0_out_WB, ResultSrc,
RegWriteData);

endmodule

```

پیاده سازی واحد کنترل:

1. منطق واحد کنترل:

در پروژه اول ما ماژول MainController را طراحی کردیم. این پروژه تغییرات بسیار کوچکی با آن دارد که در جدول بالا طراحی شده است. پس به کمک این ها این ماژول را به CU تغییر نام داده و طراحی می کنیم:

```

module CU
(
    input [6:0] operation_code,
    output reg RegWrite, AdderSrc, ALUSrc, MemWrite, PCWriteCond, PC_write,
    output reg [1:0] ResultSrc, ImmSrc, ALU_operation
);

    always @(*)
    begin
        RegWrite = 0; AdderSrc = 0; ALUSrc = 0; MemWrite = 0; PCWriteCond = 0;
        PC_write = 0;
        ResultSrc = 2'b00; ImmSrc = 2'b00; ALU_operation = 2'b00;

        case(operation_code)
            7'b0110011:
            begin
                RegWrite = 1;
                ALU_operation = 2'b10;
            end

            7'b0000011:
            begin
                ALUSrc = 1; RegWrite = 1;
                ResultSrc = 2'b01;
            end
        end
    end

```

```

7'b0010011:
begin
    ALUSrc = 1; RegWrite = 1;
    ALU_operation = 2'b11;
end

7'b1100111:
begin
    AdderSrc = 1; RegWrite = 1; PC_write = 1;
    ResultSrc = 2'b10;
end

7'b0100011:
begin
    MemWrite = 1; ALUSrc = 1;
    ImmSrc = 2'b01;
end

7'b1101111:
begin
    RegWrite = 1; PC_write = 1;
    ImmSrc = 2'b11;
    ResultSrc = 2'b10;
end

7'b1100011:
begin
    PCWriteCond = 1;
    ImmSrc = 2'b10; ALU_operation = 2'b01;
end

endcase
end
endmodule

```

ماژول های PCController (نسخه پروژه دوم) و ALUController را نیز بدون تغییر قرار می دهیم.

2. رجیسترهای واحد کنترلر:

دقیقا مانند مسیر داده واحد کنترلر، این جا هم رجیستر هایی برای ذخیره میان هر قسمت نیاز است. دقیقا مانند آن قسمت ابتدا ریز ماژول ها را طراحی می کنیم:

- Register (duplicate with data path)
- Register2bit
- Register3bit

و سپس ماژول های رجیستری میان مراحل را طراحی می کنیم:

- Controller_ID_EX
- Controller_EX_MEM
- Controller_MEM_WB

پیاده سازی HazadDetectionUnit:

این ماژول را دقیقاً همانطور که در قسمت های قبل طراحی شده بود به صورت کد های وریلاپ پیاده سازی می کنیم. دقت می کنیم که در ابتدا همه ورودی ها X هستند و خروجی ها نیز X خواهند شد و موجب اجرا نشدن کد می شود. برای رفع این مشکل ورودی ریست را به این ماژول هم می دهیم تا در ابتدا همه خروجی ها را صفر کند تا کد بتواند در همان ابتدا به درستی اجرا شود:

```
module HazadDetectionUnit
(
    input reset, PCSrcE, RegWriteM, RegWriteW,
    input [1:0] ResultSrcE,
    input [4:0] Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW,
    output reg StallF, StallD, FlushD, FlushE,
    output reg [1:0] ForwardAE, ForwardBE
);

reg lwStall;

always @(*)
begin
    StallF = 1'b0; StallD = 1'b0; FlushD = 1'b0; FlushE = 1'b0; lwStall = 1'b0;
    ForwardAE = 2'b00; ForwardBE = 2'b00;

    if(reset)
    begin
        StallF = 1'b0; StallD = 1'b0; FlushD = 1'b0; FlushE = 1'b0; lwStall =
1'b0;
        ForwardAE = 2'b00; ForwardBE = 2'b00;
    end

    if ((RegWriteM == 1'b1) && (Rs1E == RdM) && (Rs1E != 5'd0))
        ForwardAE = 2'b01;
    else if ((RegWriteW == 1'b1) && (Rs1E == RdW) && (Rs1E != 5'd0))
        ForwardAE = 2'b10;

    if ((RegWriteM == 1'b1) && (Rs2E == RdM) && (Rs2E != 5'd0))
        ForwardBE = 2'b01;
    else if ((RegWriteW == 1'b1) && (Rs2E == RdW) && (Rs2E != 5'd0))
        ForwardBE = 2'b10;

    if(ResultSrcE == 2'b01)
        lwStall = (Rs1D == RdE) || (Rs2D == RdE);

    if(lwStall == 1'b1)
```

```

begin
    StallF = 1'b1;
    StallD = 1'b1;
end

if(PCSrcE == 1'b1)
    FlushD = 1'b1;

if(PCSrcE == 1'b1 || lwStall == 1'b1)
    FlushE = 1'b1;

end

endmodule

```

پیاده سازی پردازنده (RISCV) و تاپ ماژول:

1. پیاده سازی پردازنده:

به کمک سه ماژول پیاده سازی شده در بخش های قبل و سیم کشی میان آن ها پردازنده RISCV را پیاده سازی می کنیم:

```

module RISCV
(
    input clk, reset,
    input [31:0] inst_RD, data_RD,
    output MemWriteM,
    output [31:0] inst_adr, data_adr, data_WD
);

    wire PCSrcE, AdderSrcE, RegWriteM, RegWriteW, ALUSrcE, StallF, StallD, FlushD,
    FlushE, zero;
    wire [1:0] ResultSrcE, ResultSrcW, ImmSrcD, ForwardAE, ForwardBE;
    wire [2:0] ALUFuncE;
    wire [4:0] Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW;
    wire [31:0] inst;

    DataPath stage_DataPath (clk, reset, PCSrcE, AdderSrcE, RegWriteW, ALUSrcE,
    StallF, StallD, FlushD, FlushE, ResultSrcW, ImmSrcD, ForwardAE, ForwardBE, ALUFuncE,
    inst_RD, data_RD,
                                zero, Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW, inst_adr,
    data_adr, data_WD, inst);

    Controller stage_Controller (clk, reset, FlushE, zero, inst, PCSrcE, RegWriteM,
    RegWriteW, AdderSrcE, ALUSrcE, MemWriteM, ImmSrcD, ResultSrcE, ResultSrcW,
    ALUFuncE);

```

```
    HazadDetectionUnit stage_HazadDetection (reset, PCSrcE, RegWriteM, RegWriteW,  
ResultSrcE, Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW, StallF, StallD, FlushD, FlushE,  
ForwardAE, ForwardBE);  
endmodule
```

2. پیاده سازی TopModule:

حال زمان آن رسیده است که حافظه های Data_Mem و inst_mem را هم به پردازنده اضافه کنیم و مازول سطح بالای خود را کامل کنیم:

```
module TopModule  
(  
    input clk, reset  
);  
  
    wire MemWrite;  
    wire [31:0] inst_RD, data_RD, inst_adr, data_adr, data_WD;  
  
    RISCV stage_CPU (clk, reset, inst_RD, data_RD, MemWrite, inst_adr, data_adr,  
data_WD);  
  
    Data_Mem stage_data (clk, data_adr, data_WD, MemWrite, data_RD);  
  
    inst_mem stage_inst (inst_adr, inst_RD);  
  
endmodule
```

طراحی تست:

- برنامه ای که یک آرایه 10 عنصری از اعداد صحیح علامت دار 32 بیتی را مرتب می کند.

1. انتخاب الگوریتم مرتب سازی:

○ الگوریتم مرتب سازی انتخابی (selection sort) را برای این تست انتخاب می کنیم و علت این انتخاب هم به شرح زیر است:

(1) بهینه سازی و کاهش چشمگیر مراجعات به حافظه (Memory Writes): یکی از مهمترین ویژگی های این الگوریتم مرتب سازی در این است که تعداد مراجعه به حافظه برای نوشتن (استفاده از دستور sw) بسیار کم است زیرا حداکثر $n - 1$ بار باید جابه جایی را انجام دهیم یعنی حداکثر $2n - 2$ بار از sw استفاده میکنیم (اگر تعداد عناصر را n فرض کنیم).

(2) سازگاری کامل با محدودیت های مجموعه دستورات (نبود دستورات شیفت و ضرب): با توجه به عدم وجود دستوراتی مانند slli و mul در این معماری محاسبه آدرس حافظه ($4 * \text{Index}$) کار ساده نیست. در مرتب سازی انتخابی، می توانیم به جای ضرب کردن اندیس ها، مستقیماً یک رجیستر را به عنوان آدرس پایه در نظر گرفته و با دستور `addi rd, rs, 4` به صورت متوالی در طول آرایه حرکت کنیم و آدرس کمترین عنصر را در یک رجیستر جداگانه ذخیره نگه داریم.

(3) سادگی در پیاده سازی کنترل جریان (Control Flow): در این الگوریتم، منطق شرطی بسیار ساده است. در حلقه داخلی، تنها نیاز به یک دستور پرش شرطی (مانند bge برای پرش در صورت بزرگتر یا مساوی بودن) داریم تا در صورت پیدا شدن عدد کوچکتر، مقدار مینیمم جدید ثبت شود. این ساختار سراسر باعث می شود که پیاده سازی حلقه های تو در تو با استفاده از دستورات محدود J-Type (jal) و B-Type بسیار تمیز و با کمترین میزان پرش های تو در تو در تو در اسمبلی انجام شود.

(4) مناسب بودن برای داده های با حجم کم: از آنجا که تعداد عناصر آرایه بسیار کوچک و برابر با $n = 10$ است، پیچیدگی زمانی الگوریتم که $O(n^2)$ است هیچ گونه افت عملکرد محسوسی ایجاد نمی کند. در مقابل، الگوریتم های با پیچیدگی زمانی بهتر (مثل Quick Sort یا Merge Sort) نیازمند مدیریت پیچیده پشته (Stack) و فراخوانی های بازگشتی هستند که با این مجموعه دستورات محدود، پیاده سازی آن ها به شدت سخت و غیرمنطقی است.

○ در نتیجه الگوریتم Selection Sort تعادل کاملی بین سادگی پیاده سازی (با توجه به دستورات محدود R-Type و I-Type) و کارایی (به دلیل حداقل رساندن استفاده از دستور sw) برقرار می کند و برای مرتب سازی 10 عدد صحیح علامت دار در معماری Single-Cycle RISC-V این معماری، بهترین انتخاب محسوب می شود.

2. پیاده سازی کد C++:

```
arr[10] // Assuming the array is given like this

for (int i = 0; i < n - 1; i++) {
    int minIndex = i;
    for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    swap(arr[i], arr[minIndex]);
}
```

3. تبدیل به دستورات مجاز RISC_V:

○ با استفاده از جدول جزییات سیگنال های کنترلر که func7, func3, op دستورات در آن ذکر شده است دستورات را به فرم زیر تبدیل میکنیم:

1.	addi x8, x0, 10	# x8 = n = 10
2.	addi x9, x0, 9	# x9 = n - 1 = 9
3.	add x5, x0, x0	# x5 = i = 0
4.	add x25, x0, x0	# x25 = i_offset = 0
5.	outer_loop: bge x5, x9, done	# if i >= 9, exit loop
6.	add x11, x0, x5	# minIndex = i
7.	add x26, x0, x25	# minIndex_offset = i_offset
8.	addi x6, x5, 1	# j = i + 1
9.	addi x27, x25, 4	# j_offset = i_offset + 4
10.	inner_loop: bge x6, x8, end_inner	# if j >= 10, exit inner loop
11.	lw x28, 0(x27)	# x28 = arr[j]
12.	lw x29, 0(x26)	# x29 = arr[minIndex]
13.	bge x28, x29, skip	# if arr[j] >= arr[minIndex], skip update
14.	add x11, x0, x6	# minIndex = j
15.	add x26, x0, x27	# minIndex_offset = j_offset
16.	skip: addi x6, x6, 1	# j++
17.	addi x27, x27, 4	# j_offset += 4
18.	jal x0, inner_loop	# jump to inner loop
19.	end_inner: lw x28, 0(x25)	# x28 = arr[i]
20.	lw x29, 0(x26)	# x29 = arr[minIndex]
21.	sw x29, 0(x25)	# arr[i] = old arr[minIndex]
22.	sw x28, 0(x26)	# arr[minIndex] = old arr[i]
23.	addi x5, x5, 1	# i++
24.	addi x25, x25, 4	# i_offset += 4
25.	jal x0, outer_loop	# jump to outer loop
26.	done:	

- برای مقادیر پرش با دستورات bge و jal باید دستورات را به صورت 4 حطی فرض کنیم (چون در ادامه هر دستور به 4 خط 8 بیتی تبدیل می شود) و بر اساس آن آدرس خطی که به آن باید پرش کند را پیدا کنیم که به صورت زیر می شود:

- done = +84
- end_inner = +36
- skip = +12
- inner_loop = -32
- outer_loop = -80

- نتیجه نهایی:

```

1.      addi x8, x0, 10           # x8 = n = 10
2.      addi x9, x0, 9           # x9 = n - 1 = 9
3.      add x5, x0, x0           # x5 = i = 0
4.      add x25, x0, x0          # x25 = i_offset = 0
5.  outer_loop:  bge x5, x9, 84   # if i >= 9, exit loop
6.      add x11, x0, x5          # minIndex = i
7.      add x26, x0, x25         # minIndex_offset = i_offset
8.      addi x6, x5, 1           # j = i + 1
9.      addi x27, x25, 4         # j_offset = i_offset + 4
10. inner_loop:  bge x6, x8, 36   # if j >= 10, exit inner loop
11.     lw x28, 0(x27)           # x28 = arr[j]
12.     lw x29, 0(x26)           # x29 = arr[minIndex]
13.     bge x28, x29, 12         # if arr[j] >= arr[minIndex], skip update
14.     add x11, x0, x6          # minIndex = j
15.     add x26, x0, x27         # minIndex_offset = j_offset
16.  skip:      addi x6, x6, 1    # j++
17.     addi x27, x27, 4         # j_offset += 4
18.     jal x0, -32              # jump to inner_loop
19.  end_inner:  lw x28, 0(x25)   # x28 = arr[i]
20.     lw x29, 0(x26)           # x29 = arr[minIndex]
21.     sw x29, 0(x25)           # arr[i] = old arr[minIndex]
22.     sw x28, 0(x26)           # arr[minIndex] = old arr[i]
23.     addi x5, x5, 1           # i++
24.     addi x25, x25, 4         # i_offset += 4
25.     jal x0, -80              # jump to outer_loop
26.  done:

```

4. تبدیل به دستورات باینری براساس op code های RISC_V:

```
1. 00000000101000000000010000010011
2. 00000000100100000000010010010011
3. 00000000000000000000001010110011
4. 000000000000000000000110010110011
5. 00000100100100101101101001100011
6. 00000000010100000000010110110011
7. 00000001100100000000110100110011
8. 00000000000100101000001100010011
9. 00000000010011001000110110010011
10. 00000010100000110101001001100011
11. 00000000000011011010111000000011
12. 00000000000011010010111010000011
13. 00000001110111100101011001100011
14. 00000000011000000000010110110011
15. 00000001101100000000110100110011
16. 00000000000100110000001100010011
17. 00000000010011011000110110010011
18. 11111110000111111111000001101111
19. 00000000000011001010111000000011
20. 00000000000011010010111010000011
21. 0000000111011100101000000100011
22. 0000000111001101001000000100011
23. 00000000000100101000001010010011
24. 00000000010011001000110010010011
25. 11111011000111111111000001101111
```

5. تبدیل به خطوط 8 بیتی:

- در این تبدیل به این صورت عمل میکنیم که بیت های پرازش تر دستور در خطوط پایین تر آن دستور قرار بگیر که بتوان با اتسفاده از خط زیر آنها را استخراج کرد:

```
assign RD = {mem[Adr + 3], mem[Adr + 2], mem[Adr + 1], mem[Adr]}
```

- در نتیجه داریم:

```
1. 00010011
2. 00000100
3. 10100000
4. 00000000
5. 10010011
6. 00000100
7. 10010000
8. 00000000
9. 10110011
10. 00000010
11. 00000000
12. 00000000
13. 10110011
14. 00001100
15. 00000000
16. 00000000
17. 01100011
18. 11011010
19. 10010010
20. 00000100
21. 10110011
22. 00000101
23. 01010000
24. 00000000
25. 00110011
26. 00001101
27. 10010000
28. 00000001
29. 00010011
30. 10000011
31. 00010010
32. 00000000
33. 10010011
34. 10001101
```

```
35. 01001100
36. 00000000
37. 01100011
38. 01010010
39. 10000011
40. 00000010
41. 00000011
42. 10101110
43. 00001101
44. 00000000
45. 10000011
46. 00101110
47. 00001101
48. 00000000
49. 01100011
50. 01010110
51. 11011110
52. 00000001
53. 10110011
54. 00000101
55. 01100000
56. 00000000
57. 00110011
58. 00001101
59. 10110000
60. 00000001
61. 00010011
62. 00000011
63. 00010011
64. 00000000
65. 10010011
66. 10001101
67. 01001101
68. 00000000
```

```
69. 01101111
70. 11110000
71. 00011111
72. 11111110
73. 00000011
74. 10101110
75. 00001100
76. 00000000
77. 10000011
78. 00101110
79. 00001101
80. 00000000
81. 00100011
82. 10100000
83. 11011100
84. 00000001
85. 00100011
86. 00100000
87. 11001101
88. 00000001
89. 10010011
90. 10000010
91. 00010010
92. 00000000
93. 10010011
94. 10001100
95. 01001100
96. 00000000
97. 01101111
98. 11110000
99. 00011111
100. 11111011
```

6. نوشتن آرایه 10 عنصری از اعداد صحیح:

○ برای آرایه از اعداد زیر استفاده میکنیم:

۷۷, ۱-, ۲۳, ۵۰-, ۸, ۵-, ۱۰۰, ۰, ۱۲-, ۴۵ ○

○ که فرم مکمل دو آنها به صورت زیر است:

[illegible]

- مانند قسمت دستورات الگوریتم مرتب سازی در اینجا هم هر عدد را به صورت 32 بیتی در 4 خط می نویسیم که هر خط 8 و خطوط پایین پر ارزش تر هستند:

```
1. 001011101
2. 000000000
3. 000000000
4. 000000000
5. 111101100
6. 111111111
7. 111111111
8. 111111111
9. 000000000
10. 000000000
11. 000000000
12. 000000000
13. 011100100
14. 000000000
15. 000000000
16. 000000000
17. 11111011
18. 111111111
```

```

19. 11111111
20. 11111111
21. 00001000
22. 00000000
23. 00000000
24. 00000000
25. 11001110
26. 11111111
27. 11111111
28. 11111111
29. 00010111
30. 00000000
31. 00000000
32. 00000000
33. 11111111
34. 11111111
35. 11111111
36. 11111111

```

```
37.01001101
38.00000000
39.00000000
40.00000000
```